MSIS 549

AI and GenAI for Business Applications

Foster School of Business | University of Washington

Instructor: Prof. Léonard Boussioux

Assignment 2

Agentic AI for Real-World Impact

Individual Assignment | **15 points**

Estimated Time: **4-8 hours**

⚠ **IMPORTANT:** Please read this document carefully before starting, so you understand the full scope and expectations.

Overview

In this assignment, you will design and test your own agentic workflow to solve a problem or pain point you are facing at work or in your personal life. Your system must include an ensemble of several skills/actions, and you will evaluate it using a mini-benchmark you design.

You may implement your system in any of the following ways:

- Markdown skill/agentic set designed to work together in Codex, Cloud, or any LLM assistant
- n8n workflow (or equivalent such as Google Opal, Relay.app/Stack.ai, etc.).

Your goal is to experience what it actually feels like to build and evaluate a small agentic system from scratch, then assess whether it is truly useful in practice. If relevant, you may repurpose your course project idea for HW2.

Learning Objectives

By completing this assignment, you will:

- Design a real-world agentic workflow from scratch

- Connect multiple skills/agents into a coherent system
- Practice prompt design and improvement based on actual usage
- Build and run a rigorous mini benchmark to evaluate performance
- Critically assess usefulness, limitations, and tradeoffs

A Note on Why This Assignment Is Open-Ended

This homework is intentionally broad. There are countless workflows one could build, and I made a deliberate design choice to let you pursue whichever one matters most to you. The alternative — having every student replicate the same predefined workflow — would certainly teach that one specific thing, but it would also constrain the path so much that the lessons become harder to generalize. At its core, working with AI is about building mental models: understanding how AI succeeds, where it struggles, and how to orchestrate it effectively. By choosing your own workflow, you will each encounter different challenges and different difficulties — and that is exactly the point.

If I were to attempt every one of your workflows myself, I would run into bottlenecks, too. In fact, as an exercise, I tried to build some of the workflows you suggested in class, and it wasn't always easy. I had to iterate, abandon some approaches, and try alternatives before learning which ones were truly suitable. That process of discovery is what this assignment is designed to teach. Whenever you have a little time to spare, I encourage you to keep experimenting on your own — it compounds quickly. I also highly recommend reading this blog post by Ethan Mollick: Management as AI Superpower.

The Assignment

Part 1: Choose a Real Workflow

Select a workflow that actually matters to you or your environment. It must be real, not fictional, and you should be able to run it at least twice on real inputs.

Examples include:

- Market research for a product or competitor
- Drafting and iterating LinkedIn posts
- Writing and editing a blog post
- Customer interview synthesis
- Competitive analysis and differentiation
- Sales outreach drafting and personalization
- Data cleaning and reporting summary
- Recruiting pipeline screening summary

Here are your own ideas as a class in a dashboard: https://skillforall.lovable.app

Part 2: Build Your Agentic System

You must choose one of the two paths below. Your system must run end-to-end and include at least three distinct components.

Path A: Skills Pack (Markdown files)

Create a reusable set of skills as Markdown files (2-5 skills recommended) that can work standalone when installed. You may use Claude, OpenAI, or any other LLM to define and test the skills.

✨ **Bonus (1 point):** include an MCP tool.

Each skill file can typically include:

- Skill name and purpose
- Inputs and outputs
- Step-by-step behavior
- Constraints and failure modes
- Example usage

You will find an example of a skill markdown in Appendix.
You can use the built-in skill creator in your preferred model (Claude, Codex, and Gemini CLI all include one). Simply activate it and describe your intent in natural language — the model will interview you to fill in the details. As a best practice, define what success looks like, what context the skill should check, and provide template answers where possible.

Note: Skills and their integration with models are still very new, so occasional failures are expected. This is part of the learning process. If something goes wrong, it may mean the workflow isn't well suited for skills yet, or that the skill needs to be redesigned to handle more edge cases. Your grade is based on the quality of your attempt, your adherence to good practices, and your ability to reflect on outcomes — not on whether everything works perfectly. Keep in mind that not every workflow is a good fit for skills today (though this is improving rapidly). For example, suppose you want a system that checks your calendar for deadlines and sends you a reminder email. You could build this with skills and MCPs, but it would only run while your computer is on. If you need it to run automatically in the background — even when your machine is offline — a tool like n8n is a better choice, since it can deploy to a dedicated server.

Before choosing your approach, ask yourself: Do I want a workflow I launch on demand from my own machine? Or do I need something fully automated that runs independently and can be shared with others via a link? Skills work well for personal, on-demand workflows, but sharing them across multiple computers is still somewhat limited. This is improving quickly, so consider your needs today and choose the path that fits best.

Path B: n8n or SDK Workflow

Build the same idea using n8n or any SDK (Google Opal, OpenAI Agent Builder, Relay.app, Stack.ai, etc.).

Your workflow must include:

- At least 3 reusable components or tools
- Clear orchestration logic (routing, sequencing, or tool-calling)

- Basic error handling or fallback behavior

n8n offers a free trial along with extensive tutorials and a built-in AI-assisted builder. Setting up complex workflows in n8n can take some effort, but it is a great opportunity to learn by trial and error — and you can always ask your favorite LLM for help, as they are all knowledgeable about n8n. You will likely need to connect an API key, which may incur small costs, though n8n includes free credits during the trial period so you can test models at no charge.

Two solid alternatives are the OpenAI Agent Builder (which also requires an API key) and Google Opal, which provides free usage for testing. All three platforms offer templates and documentation to help you get started. Of the three, n8n has the broadest set of capabilities. If you prefer a clean, easy-to-navigate interface, Google Opal is worth trying. Ultimately, the best choice depends on your personal preference and what fits your workflow.

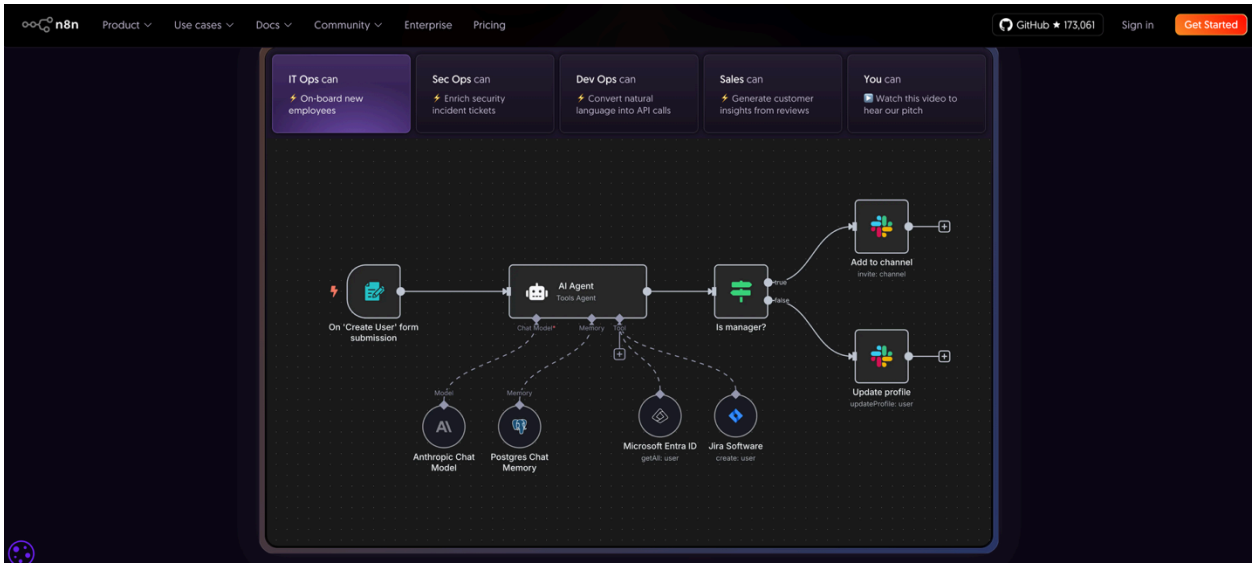

Figure 1: Example workflow in n8n with AI agent orchestration.

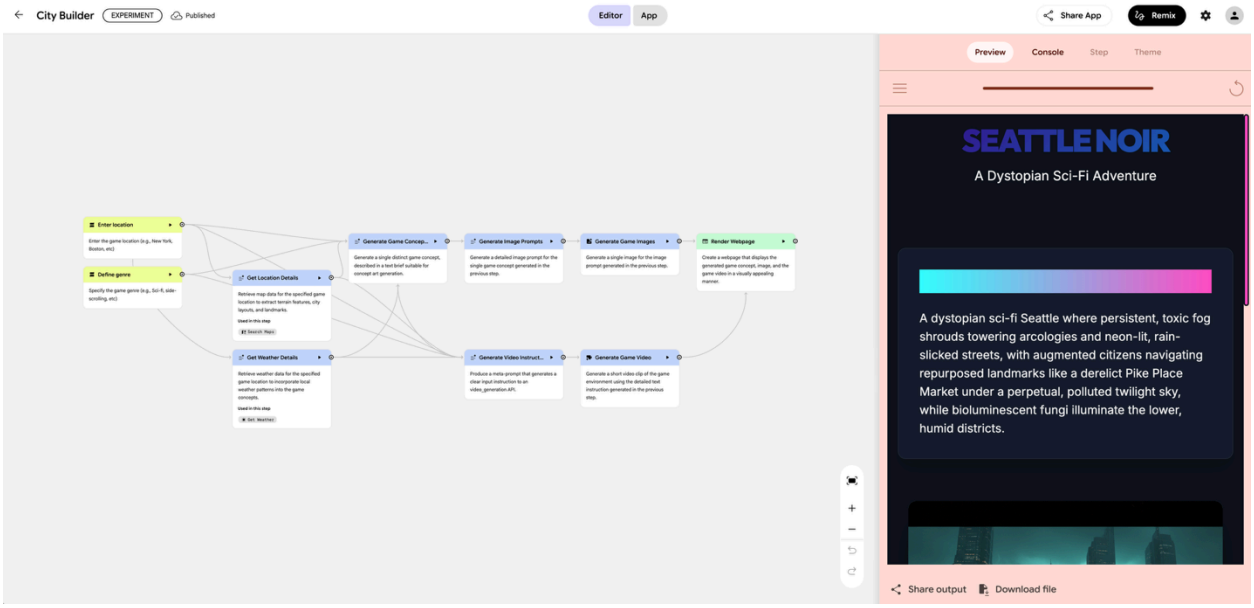

Figure 2: Google Opal visual workflow editor with live preview.

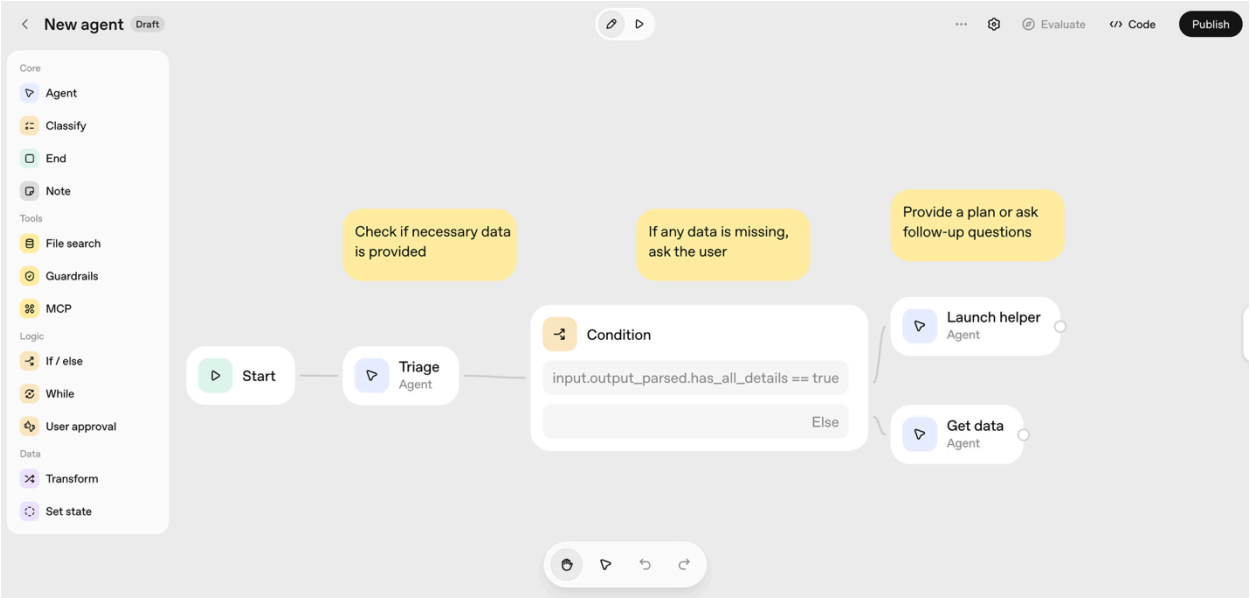

Figure 3: OpenAI Agent Builder with triage routing and conditional logic.

Part 3: Real Usage + Iteration

Run your system on at least two real tasks and keep track of:

- The inputs you used
- The outputs produced
- What you changed after the first few runs
- Whether the system helped or not
- Where it failed, hallucinated, or slowed you down

Part 4: Rigorous Mini Benchmark

You must design a mini-benchmark to test your system's performance. This is one of the most important parts for a successful agent.

Your benchmark can include:

- 2+ test cases that reflect the real task
- At least one baseline comparison (single-prompt LLM or your own manual process) if relevant for your workflow (e.g., generating a blogpost, a linkedin post, a cover letter).
- Metrics you define (accuracy, time saved, hallucination rate, clarity, actionability, etc.)
- A scoring rubric (0-1 or 1-5 scale) with brief anchors
- Summary results and failure analysis if any
- At least 2 concrete examples in your tutorial write-up (show inputs and outputs)

You may use any evaluation method you prefer (human scoring, LLM-as-judge, automated checks, or a mix). There is no penalty for the method you choose, as long as it is clear and consistent.

📋 **Rigor Guidelines**

- Define success criteria before testing
- Include at least one edge case and one ambiguous case that a user could input
- Freeze prompts and settings across all test cases
- Report aggregate results and the worst failure
- Provide inputs and outputs so the test is reproducible
- More tips in Appendix.

Part 5. [Bonus] Record a Demo

Record a demo video showing your system running end-to-end on a real input. The demo should make it easy for a learner to see what the user provides, what the system does, and the final output.
Include, at minimum:

- The input you provide to the system
- A quick view of the workflow/agent orchestration (e.g., how skills/agents are invoked)
- The final output produced
- One brief mention of how you would evaluate success (e.g., metric or criterion)

Any screen-recording tool is fine (Panopto, Riverside, Loom, Zoom, etc.). I am not asking for perfection in this video; any reasonable attempt is fine (i.e., don't spend more than 30-60 min on it). Pro tip: Practice the demo for smoothness and then record. No need for any special editing, but you should feel comfortable having your classmates (or anyone you'd share the video with) watching it.

Deliverables

Submit the following via Canvas:

System Artifact	Shareable link to your agents or skills. A github repo is a good practice but not required.
Tutorial Write-up	PDF with links to your system artifact above. A replicable tutorial covering problem statement, system design, prompt documentation, building process, real usage, benchmark methodology and findings, and reflection. Include as many figures as needed.
Benchmark Appendix	Table of test cases, outputs, and scores
[Optional] Demo Recording	2-5 minutes showing your system in use with URL link on Canvas + embedded in your Tutorial write-up

Tutorial Write-Up Guidelines

Your main written deliverable is a tutorial in PDF format. Write it so that someone who has never seen your workflow could follow your instructions and successfully replicate your agent from scratch. Think of every bottleneck you encountered, every question you had to answer along the way, and every decision that was not obvious — then make sure your tutorial addresses all of them.

Start with the problem. Begin your tutorial by clearly describing what you are trying to solve: the pain point, the current status quo, and why an agentic workflow is a promising approach. This gives your reader the context they need before diving into the technical details. Briefly outline what your workflow will do, and what the tech stack is for this tutorial.

Document your system, depending on your path:

If you chose Path A (Skills Pack):

- Provide a brief overview of each skill: its name, its purpose, and a one- or two-sentence summary of what it does.
- Include the exact prompts or instructions embedded in each skill file (in Appendix or inside the tutorial if relevant for discussion).
- Document at least one prompt iteration: what you changed, why you changed it, and whether it improved the output.
- Offer a brief critique of your prompt quality (clarity, constraints, tone, and evaluation criteria).

If you chose Path B (n8n / SDK Workflow):

- Include a screenshot or diagram of your workflow so the reader can see the overall structure at a glance.
- Document every prompt you used within the workflow nodes, so that someone could recreate the exact same pipeline.
- Document at least one prompt iteration: what you changed, why you changed it, and whether it improved the output.

- Offer a brief critique of your prompt quality (clarity, constraints, tone, and evaluation criteria).

Document your building process. Feel free to include as many figures, screenshots, and details as you like. For example: which tools and LLMs did you use? What prompts did you give to build or refine the system? How long did each step take? What were the frustrating parts? The more transparent you are, the more useful your tutorial will be.

Benchmarking. Dedicate a meaningful portion of your tutorial to explaining how you tested your system. Describe how you designed your benchmark, how you ran it, and what the results tell you about the quality of your agent. Refer to Appendix 2 for suggested evaluation methods.

Reflection Prompts

Here are elements you can discuss in your write-up (choose on what is relevant):

- Why this workflow, and why it matters
- What worked well vs. what did not
- How your prompts evolved and why
- What your benchmark revealed
- Would you keep using this system? Why or why not

Sharing your work. Include direct links to your system so that someone could replicate or try it (Canvas will also ask you for these links separately). For example, you could create a GitHub repository, host your files on Google Drive, or share a direct link to your workflow. After the course ends, I will invite you to optionally open-source your tutorial and code so we can showcase our collective creations and let others — both classmates and beyond — benefit from your work.

Grading Rubric

15 points total + 1 bonus

Category	Excellent	Satisfactory	Needs Work
Problem Selection **1 pt**	Workflow is genuinely meaningful to you/your context with clear justification	Real workflow but connection to your context is unclear or weak	Fictional or trivial workflow with no real-world grounding
System Design **4 pts**	Distinct agents/nodes/skills with clear roles, logical flow, and thoughtful integration	Agents present but roles overlap or flow is unclear	Fewer than 2 agents/skills/nodes or agents are essentially the same prompt repeated
Implementation Quality **3 pts**	Runs end-to-end reliably, outputs are usable and well-formatted. Prompts are well-structured with	Runs with minor issues or outputs require manual cleanup. Prompts provided but iteration is	Frequent failures, unusable outputs, or incomplete implementation. Prompts

	at least one meaningful iteration and honest critique.	cosmetic or critique is superficial	missing, no iteration, or no self-critique
Real Usage + Iteration **2 pts**	Clear evidence of 2+ real runs with documented changes and reflections	Evidence of real usage but iteration or reflection is shallow	No evidence of actual use or only hypothetical scenarios
Tutorial writing overall quality **3 pts**	Tutorial is clear, well-structured, and genuinely replicable — a reader could rebuild the agent by following it. Covers problem statement, building process, prompts, and benchmarking with enough detail and transparency (figures, links, honest discussion of bottlenecks).	Tutorial covers the main topics but some steps are vague or missing detail. A reader could understand the approach but would struggle to replicate it without guessing.	Tutorial is superficial, disorganized, or missing key sections. Little effort to make the work replicable or to document the building process honestly.
Benchmark Rigor **2 pts**	2+ test cases, clear metrics, edge cases, and insightful failure analysis	Test cases present but missing baseline, edge cases, or analysis lacks depth	Minimal testing, no baseline, or results not reproducible
Bonus: Demo Video **+ 1 pt**	Clear, well-paced demo showing the full end-to-end workflow on a real input. Easy to follow what is provided, what the system does, and the final output.	Demo covers the workflow but is hard to follow, skips steps, or does not clearly show inputs and outputs.	No video submitted, or video is too short or unclear to understand what the system does.
Bonus: MCP Tool **+1 pt**	Working MCP tool that meaningfully extends system capabilities		

Resources

Recommended learning materials:

Agent Skills with Anthropic (DeepLearning.AI) — ***Highly recommended***
Short, hands-on course that walks through practical agent design and tool use.

Claude Skills Documentation

Official guidance on skill structure, definitions, and best practices for Claude.

Complete Guide to Building Skills for Claude (PDF)
A detailed playbook with examples, patterns, and dos/don'ts for robust skills.

OpenAI Codex Skills Documentation
Official docs for building skill files and composing multi-skill systems in Codex.

Claude Sub-Agents Documentation
Concepts and patterns for multi-agent setups and task delegation.

Appendix 1: Skill example as .MD

---
name: msis-homework-generator
description: Generate new student homework assignments for MSIS 549 and MSIS 522 using the current syllabus and HW1 as the style and requirements baseline. Use when asked to draft a new homework, assignment prompt, grading rubric, deliverables list, or reflection questions from a short instructor prompt.

MSIS Homework Generator

Overview

Generate student-facing homework prompts aligned to the syllabus modules and learning objectives. Match the tone, structure, and rubric style of HW1 while adapting to the new topic and requirements.

Workflow

1. Intake the quick prompt.
- Extract: course (MSIS 549 vs MSIS 522), homework number, topic/module, intended skills, estimated time, due date (if any), deliverable types, and grading weight or points.
- If any of these are missing, proceed with a draft and include an explicit Assumptions section plus 2-4 concise clarification questions at the end.

2. Load course context.
- Read the current syllabus and HW1 prompt from the workspace using the paths in `references/course-files.md`.
- If either file is missing, ask the user for the correct path before drafting.

3. Derive constraints and style anchors.
- Capture the relevant module, learning objectives, and expected rigor from the syllabus.
- Mirror HW1 structure: Overview, Learning Objectives, Assignment parts, Testing/Feedback (if relevant), optional research component, reflection blog post (if relevant), Deliverables, Rubric.
- Keep requirements measurable and student-friendly.

4. Draft the assignment.
- Use the template in `references/assignment-template.md`.
- Make deliverables explicit and graded.
- Include a rubric table whose points sum to the requested total (default: 15 points).
- Ensure the workload matches the estimated time window.

5. Quality checks.
- Verify alignment with the specified module and skills.
- Ensure each requirement is testable and not ambiguous.
- Avoid nested bullet lists unless absolutely necessary.
- If assumptions were made, highlight them clearly.

Output Format

Produce a single Markdown assignment handout that students can read directly. Include a short Assumptions section only when needed.

Resources

- `references/course-files.md` for the syllabus and HW1 locations.
- `references/assignment-template.md` for the draft skeleton.

Appendix 2: Benchmark Methods

There is no single industry standard for evaluating agentic systems. Most teams use a combination of methods depending on scale, risk, and time. Below are common approaches you can choose from.

1) Human Rubric Scoring

Small test set, scored by a human using a clear rubric with anchors. This is often treated as the gold standard for quality on a limited set of cases.

2) LLM-as-Judge

Use an LLM to score outputs against a rubric. Often used for rapid regression testing, but should be calibrated against human judgment.

3) Baseline Comparison (A/B)

Compare your system against a baseline such as a single-prompt LLM, manual workflow, or previous version. Focuses on relative improvement.

4) Regression Test Suites

Fixed test cases that you re-run after changes to prompts, tools, or workflows to check for drift.

5) Edge-Case Testing

Intentionally difficult or ambiguous inputs to surface failure modes and safety issues early.

6) Operational Metrics

Latency, cost per output, error rate, escalation rate, and other operational stats.

For this course, any of the above methods is acceptable. Choose the option that best fits your workflow and explain your choice clearly.

Questions?

leobix@uw.edu • archer95@uw.edu • am007@uw.edu